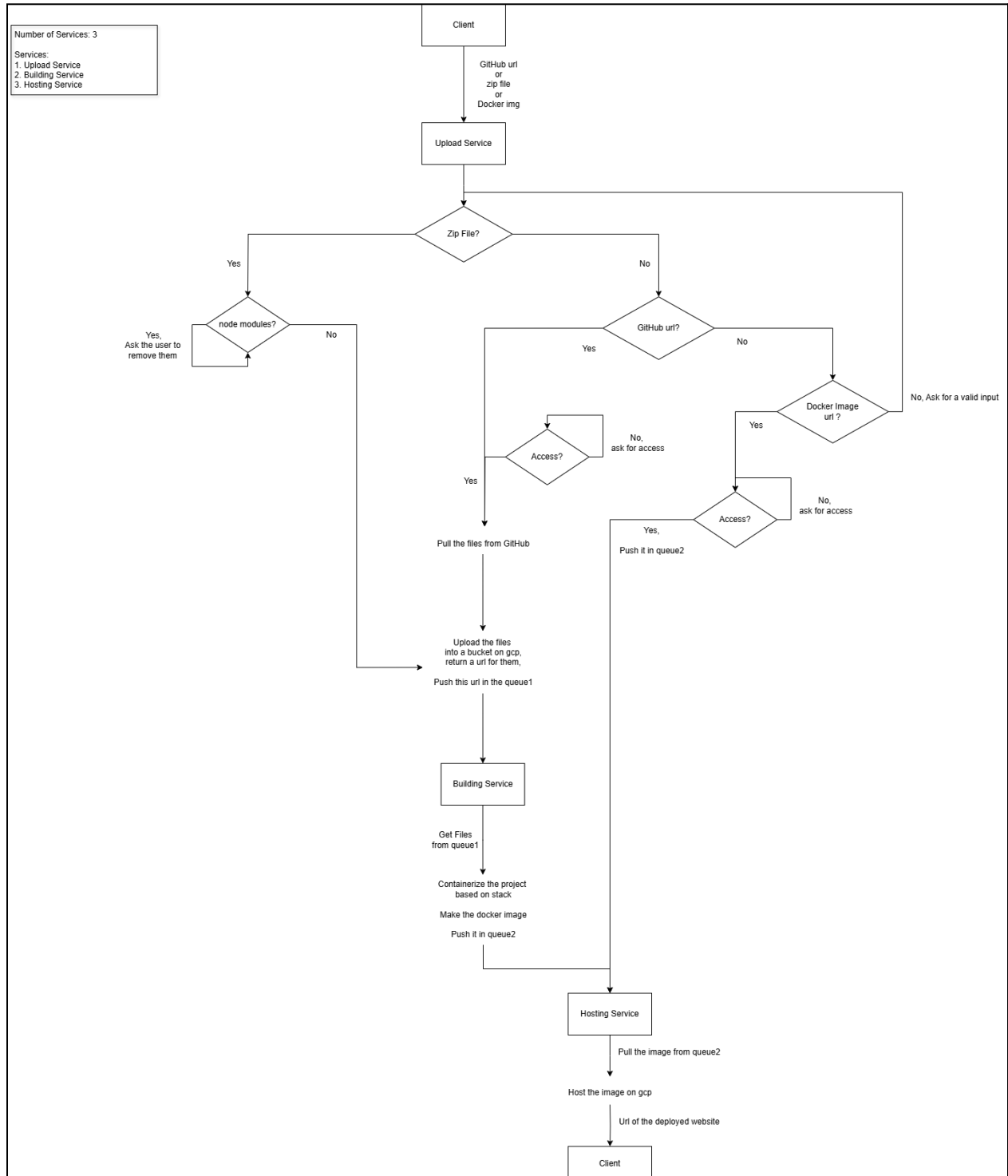


Design Document

Group Number : 22

High Level Design:



Our High Level Design consists of **three microservices** and **two queues** :

1. **Upload Service:**

Responsible for uploading the codebase of the repository in the cases of *github_url* / *zip_file*, then pushes the url of the uploaded project in the **build queue**. For the case of *docker_image*, it directly pushes the docker image in the **hosting queue**.

2. **Build Queue:**

Responsible for getting the projects that are to be built from **upload Service**, and giving them to the **build Service**.

3. **Build Service:**

Responsible for building the docker image of the project, and then pushing this docker image to the docker hub. Also enqueues the docker images in the **hosting queue**.

4. **Hosting Queue:**

Responsible for getting the projects' docker image from the **build service**, then giving them to the **host service**. Also can get the docker image from the **upload service** if a docker image is given by the user.

5. **Hosting Service:**

Responsible for hosting the project's docker image on **Google Cloud Run**, provides the url through which the application can be accessed.

This design is following the **Layered Architecture** in which:

1. Layer 1: Upload Service
2. Layer 2: Build Service
3. Layer 3: Host Service

These layers communicate with each other using the Build and the Host Queues in between them.

This **Layered Architecture** provides us with the ability to **horizontally scale** the deployment process in case of high demand.

The upload, build and host services are kept separately because of the following reasons:

1. The Upload and Host Services are lightweight but the Build process is heavy.
2. If any of the Upload / Host Service is merged with Build Process then the building process of the project can become the bottleneck.
3. To avoid this bottleneck, the build process can have multiple copies. So the upload and host services can have lesser copies as compared to the build process when we encounter high traffic.
4. This saves us resources and provides high availability.

The use of **Queues**:

1. The queues are made so that the processes can communicate with each other.
2. The queues are implemented in such a way that race conditions and data inconsistency are taken care of internally.
3. For a scenario of 3 Upload processes and 6 Build Process copies, we'll still have just 1 Build Queue which will receive the urls from Upload Services, and give them to the Build Processes. Since race conditions are taken care of, we can say for sure that each build process will always receive a unique project to build and no two processes are building the same project at any instant.
4. The same can be said for Hosting Queue, as it can happen that multiple Build Services deliver the docker images at the same time, this queue ensures that no docker image is missed and all are provided to the Host Services. Also it ensures that no 2 Host Processes are hosting the same project.

As a part of mini project, we worked upon the task:

"Implement any cloud computing related aspect and show the performance improvement over non-cloud setup by generating experimental results from the system implemented by you."

We've done the implementation of Queue Service over our local system for comparison with the Queue Service on Google Cloud

Non Cloud:

```
Completed enqueueing 1000 messages  
Completed dequeuing 1000 messages  
Execution Time: 14.884s
```

Cloud:

```
Completed enqueueing 1000 messages  
Completed dequeuing 1000 messages  
Execution Time: 12.480s
```

So for enqueueing and dequeuing **1000** messages we've got the above results. These results show an improvement of **2.404s i.e. 16.15%**.

Reason:

The processing of messages that are enqueued and dequeued is limited by the number of cycles that our CPU is going through per second.

So the CPU that is given to the queue service on cloud might be having more cycles per second than our local machine's CPU.